

| Pourquoi Terraform ?

Les bénéfices qui expliquent son adoption



1 Infrastructure as Code

L'infrastructure est décrite dans du code, versionnable et relisible.



2 Reproductibilité

Le même code permet de recréer le même environnement de façon fiable.



3 Multi-cloud

Un même outil pour piloter Azure, AWS, GCP et bien d'autres services.



4 Automatisation & CI/CD

S'intègre facilement aux pipelines pour déployer plus vite et avec moins d'erreurs.



En bref : Terraform simplifie le provisioning d'infrastructure de manière déclarative, prévisible et industrialisable.

Concepts de base de Terraform

Les briques essentielles à comprendre



HCL

Le langage déclaratif utilisé pour écrire l'infrastructure.



Providers

Les plugins qui relient Terraform aux plateformes et APIs.



Resources

Les objets que Terraform crée, modifie ou supprime.



Data Sources

Les informations lues depuis l'existant, sans le créer.



Variables & Outputs

Les entrées et sorties qui rendent le code flexible.



Modules

La réutilisation de blocs d'infrastructure standardisés.



Idée clé : Terraform assemble ces concepts pour décrire, lire, créer et réutiliser l'infrastructure.

Architecture & Fonctionnement

Le modèle simple mais puissant de Terraform

1

Écrire

Définition de l'infrastructure en code (HCL)



2

Planifier

Terraform calcule le plan d'exécution et affiche les changements



3

Appliquer

Terraform applique les changements dans le bon ordre



4

État

L'état stocke la représentation de l'infrastructure gérée



5

Synchronisation

Terraform compare la configuration désirée avec l'état et corrige les écarts

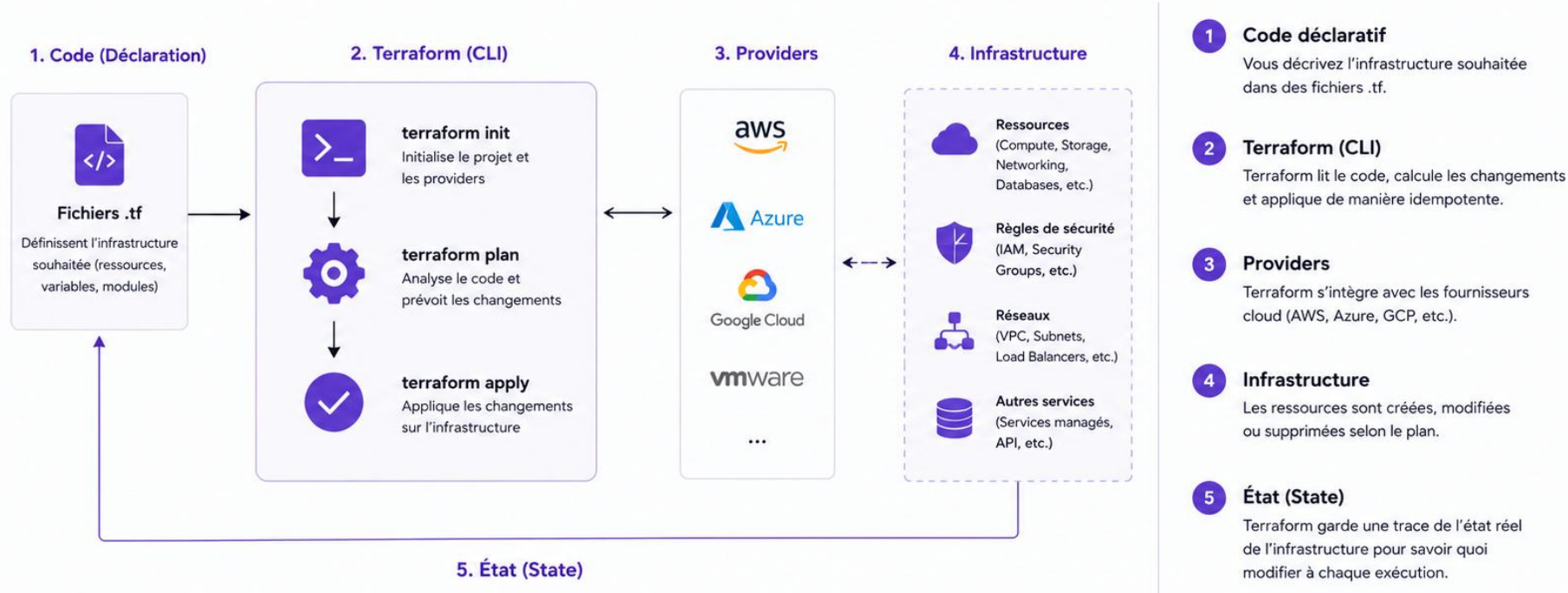


À retenir

Terraform suit un cycle déclaratif : vous décrivez l'état final souhaité, il calcule et applique les actions pour y parvenir et maintient la cohérence dans le temps.

Architecture & Fonctionnement

Terraform : un outil déclaratif pour gérer l'infrastructure as Code



Idempotent, Versionné, Collaboratif

Chaque exécution de Terraform converge vers l'état désiré, en toute sécurité et reproductibilité.

Providers

Le pont entre Terraform et les plateformes



Plugin spécialisé



Gère l'authentification



Expose ressources et data sources



À retenir : sans provider, Terraform ne sait pas dialoguer avec une plateforme.

Resources

Les objets que Terraform gère

TYPE
= type de ressource

NAME
= nom local Terraform

```
resource "TYPE" "NAME" {  
    argument = valeur  
}
```



VM / instance



Réseau / VNet



Storage / base de données

- Crée, modifie ou supprime
- Peut dépendre d'autres ressources
- Est suivi dans le state



À retenir : un resource représente un élément concret de l'infrastructure.

Data Sources

Lire l'existant sans le créer



```
data "azurerm_resource_group" "rg" {  
  name = "rg-prod"  
}
```



Infrastructure
existante

lecture



Terraform



Récupère des IDs



Réutilise l'existant



Évite la duplication



À retenir : une data source lit une information déjà présente dans la plateforme.

Variables & Outputs

Paramétrer et exposer les informations utiles



Variables

```
variable "location" {  
    default = "France Central"  
}
```

- Rendent le code réutilisable
- Peuvent être typées et validées



Outputs

```
output "vm_ip" {  
    value = azurerm_linux_virtual_machine.vm.public_ip_address  
}
```

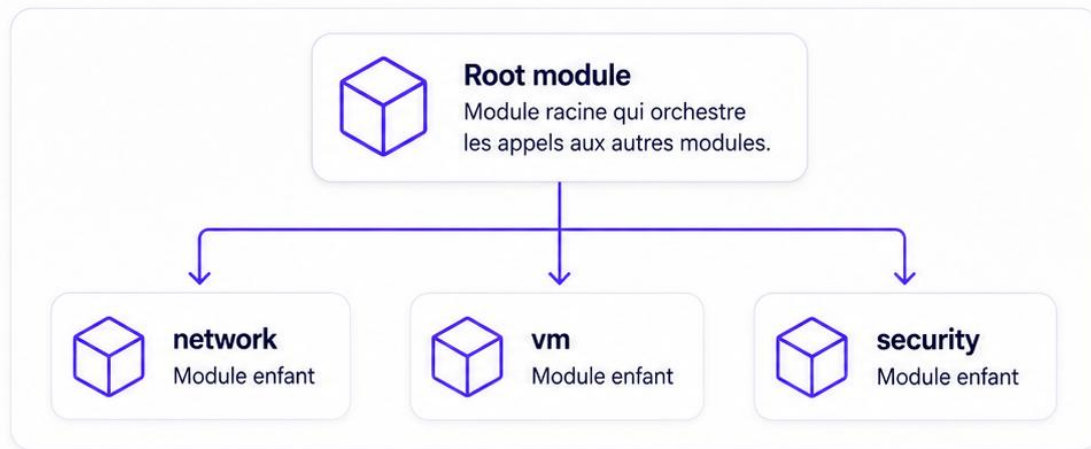
- Expose une valeur utile
- Pratique pour l'exploitation et l'intégration



À retenir : les variables font entrer des paramètres, les outputs font sortir des résultats.

Modules

Réutiliser des blocs d'infrastructure



```
module "network" {  
  source = "../modules/network"  
}
```



Réduction de la duplication

Évitez de copier-coller les mêmes blocs d'infrastructure.



Standardisation

Appliquez des bonnes pratiques et des conventions communes.



Maintenance plus simple

Une modification dans le module profite à tous les usages.



À retenir : un module permet de factoriser et réutiliser une même logique Terraform.

Les limites de Terraform



Ce que Terraform ne fait pas, ou fait moins bien



1 Pas un outil de configuration

Il crée l'infrastructure, mais ne remplace pas Ansible pour configurer finement les serveurs.



2 Gestion de l'état sensible

Le fichier state doit être protégé, sauvegardé et verrouillé pour éviter conflits et fuites.



3 Orchestration limitée

Pour les workflows applicatifs complexes, Terraform n'est pas l'outil le plus adapté.



4 Certaines fonctions restent natives

Auto-scaling avancé, déploiements blue/green ou runtime applicatif sont souvent mieux gérés ailleurs.



À retenir : Terraform excelle pour provisionner l'infrastructure, pas pour tout faire autour.

Les outils qui complètent Terraform



Chaque outil couvre une limite précise



Ansible

Configuration des serveurs
et post-provisioning.



Packer

Création d'images machines
prêtes à l'emploi.



Kubernetes / Helm

Déploiement et orchestration
des applications conteneurisées.



Vault

Gestion des secrets et des
données sensibles.



Terragrunt

Structuration multi-environnements
et réutilisation Terraform.



Bonne pratique : Terraform s'intègre à un écosystème d'outils spécialisés
plutôt que de chercher à tout couvrir seul.